DEVOPS MASTER PRACTICAL HANDBOOK

Quick exam-revision guide • Windows + web-console workflows

Practical	Topic
1	Agile project lifecycle using Jira + GitHub
2	Git version control and collaboration
3	Jenkins installation and Hello World build
4	Jenkins + GitHub CI/CD integration
5	Kubernetes container orchestration
6	Docker containerization
7	Grafana application monitoring
8	AWS Lambda serverless function

How to use: Revise the Aim, tools, procedure, commands, expected result, viva and conclusion for each practical. Replace example project names, IDs, and URLs with the ones assigned by your instructor.

Important: Practical 5 (Kubernetes) depends on a working container runtime. Practical 6 (Docker) may be completed independently once Docker Engine is running. Follow your college's rules about permitted notes or reference material during assessments.

Practical 1: Agile Project Lifecycle using Jira with DevOps Integration

Aim

Aim: To plan and manage an Agile software project using Jira and connect development work with a GitHub repository.

Software / Tools Required

Jira, GitHub account, web browser, Git.

Procedure

1. Create a Scrum project/space in Jira and open its backlog.
2. Create a task or user story (example: Design homepage) and assign it to a sprint.
3. Add team members/collaborators and set priorities or due dates as required.
4. Connect the Jira project/organization to GitHub using the available GitHub integration.
5. Create or select a GitHub repository (example used: experiment02).
6. Create a feature branch (example: SCRUM-1-design-homepage), make a change, commit and push it.
7. Open a Pull Request (PR) to merge the feature branch into the main branch; review and merge it.
8. Check that the Jira issue shows the linked branch, commits or pull request.

Commands / Configuration

```
Example Git workflow:
git checkout -b SCRUM-1-design-homepage
git add .
git commit -m "Design homepage"
git push -u origin SCRUM-1-design-homepage
# Create and merge the Pull Request on GitHub
```

Expected Output / Result

Jira displays project tasks and sprint progress. GitHub shows the feature branch and merged PR. In the completed exercise, PR #1 for SCRUM-1-design-homepage was merged and closed.

Important Viva Questions

- **What is Agile?** An iterative approach that delivers work in small increments.
- **What is a Scrum sprint?** A fixed time period in which a team completes selected backlog work.
- **What is a backlog?** A prioritized list of work items.
- **What is a pull request?** A request to review and merge code changes from one branch into another.
- **Why integrate Jira and GitHub?** To connect planning issues with code branches, commits and pull requests.

Conclusion

The project was planned in Jira and development activity was linked with GitHub, demonstrating an Agile workflow.

Practical 2: Version Control using Git and Collaboration

Aim

Aim: To perform Git repository operations, branching, merging, cloning and pushing changes to GitHub.

Software / Tools Required

Git for Windows, PowerShell/CMD, GitHub account, internet.

Procedure

1. Check Git installation and create a project folder.

2. Initialize a repository and create a file.

3. Stage and commit the file.

4. Create a feature branch, add another file and commit it.

5. Switch to main/master and merge the feature branch.

6. Create a GitHub repository, connect it as origin and push the main branch.

7. Clone the repository into another folder, add a file, commit and push it.

Commands / Configuration

```
git --version
mkdir Git-Practical
cd Git-Practical
git init
echo Hello > index.html
git add index.html
git commit -m "First commit"
git branch feature
git switch feature
echo About > about.html
git add about.html
git commit -m "Added about page"
git switch master
git merge feature
git branch -M main
git remote add origin https://github.com/USERNAME/Git-Practical.git
git push -u origin main
git clone https://github.com/USERNAME/Git-Practical.git
# In the cloned folder:
echo Contact > contact.html
git add contact.html
git commit -m "Added contact page"
git push origin main
```

Expected Output / Result

Git reports successful commits, branch merge, push and clone. The repository contains index.html, about.html and contact.html. If origin already exists, use git remote set-url origin URL instead of adding it again.

Important Viva Questions

- **What is Git?** A distributed version control system.
- **What is a commit?** A saved snapshot of staged changes.
- **What is a branch?** An independent line of development.
- **What is merge?** Combining changes from one branch into another.
- **What is clone?** Creating a local copy of a remote repository.

Conclusion

Git operations and GitHub collaboration were performed successfully.

Practical 3: Jenkins Installation and Hello World Build

Aim

Aim: To install and configure Jenkins and execute a simple build job.

Software / Tools Required

Java JDK (JDK 21 was installed), Jenkins, web browser.

Procedure

1. Install a supported JDK and verify it using java -version.

2. Install Jenkins using the Windows installer; configure it as a Windows service and use the default port 8080.

3. Open http://localhost:8080 and unlock Jenkins using the initial administrator password.

4. Install suggested plugins and create the first administrator account.

5. From the dashboard select New Item, enter Hello-World and choose Freestyle project.

6. Under Build Steps, select Execute Windows batch command and enter echo Hello World.

7. Save, select Build Now, then open the build's Console Output.

Commands / Configuration

```
java -version
# Jenkins job: Execute Windows batch command
echo Hello World
```

Expected Output / Result

Console Output displays Hello World and ends with Finished: SUCCESS. Jenkins dashboard shows the successful build.

Important Viva Questions

- **What is Jenkins?** An automation server used for building, testing and delivering software.
- **What is a job?** A configured task Jenkins executes.
- **What is a build?** An execution of a job.
- **What is a plugin?** An extension that adds functionality to Jenkins.
- **What is CI?** Continuous Integration: frequently integrating code and validating it through automated builds/tests.

Conclusion

Jenkins was installed and a Freestyle job executed successfully.

Practical 4: Jenkins and GitHub Integration (CI/CD)

Aim

Aim: To connect Jenkins with GitHub and automate a build/test workflow using source code from a repository.

Software / Tools Required

Jenkins, Git, GitHub repository, web browser.

Procedure

1. Create a new Freestyle project named GitHub-Integration.

2. Under Source Code Management select Git and enter the repository URL.

3. Set the branch specifier to */main. For a public repository, credentials may be left as None.

4. Under Build Steps select Execute Windows batch command.

5. Enter simple build/test commands and save the job.

6. Select Build Now and inspect Console Output.

7. Optional: configure Build Triggers > Poll SCM with schedule H/5 * * * * to check for changes approximately every five minutes.

Commands / Configuration

```
Example batch build step:
echo Building GitHub Project
dir
echo Testing completed successfully

Optional Poll SCM schedule:
H/5 * * * *
```

Expected Output / Result

Jenkins checks out the GitHub repository, lists its files, prints the build/test messages and finishes with SUCCESS. Poll SCM periodically checks for new commits; it is not a guaranteed instant trigger.

Important Viva Questions

- **What is CI/CD?** CI automates integration and validation; CD automates delivery or deployment.
- **What is SCM?** Source Code Management; it tracks and manages code changes.
- **Why connect Jenkins to GitHub?** So Jenkins can retrieve source code and build/test it.
- **What is Poll SCM?** A Jenkins trigger that periodically checks the repository for changes.
- **What is a pipeline?** A sequence of automated stages such as build, test and deploy.

Conclusion

Jenkins successfully retrieved the GitHub project and executed the configured build/test commands.

Practical 5: Kubernetes Container Orchestration

Aim

Aim: To deploy and manage containerized applications using Kubernetes.

Software / Tools Required

Docker Engine or another supported container runtime, Kubernetes, kubectl, Minikube (or Docker Desktop Kubernetes).

Procedure

1. Ensure the container runtime is running.
2. Install kubectl and Minikube if using the Minikube approach.
3. Start a local Kubernetes cluster.
4. Create a deployment using an image such as nginx.
5. Check deployment and pod status.
6. Expose the deployment as a NodePort service and access it using Minikube's service command.
7. Delete the service and deployment after the demonstration if required.

Commands / Configuration

```
minikube start --driver=docker
kubectl create deployment myapp --image=nginx
kubectl get deployments
kubectl get pods
kubectl expose deployment myapp --type=NodePort --port=80
kubectl get services
minikube service myapp
# Cleanup:
kubectl delete service myapp
kubectl delete deployment myapp
```

Expected Output / Result

Minikube starts a local cluster; kubectl shows the deployment and running pod; a NodePort service exposes the Nginx application.

Important Viva Questions

- **What is Kubernetes?** An open-source platform for orchestrating containerized applications.
- **What is a Pod?** The smallest deployable unit in Kubernetes; it can contain one or more containers.
- **What is a Deployment?** A controller that manages application Pods and replicas.
- **What is kubectl?** The command-line tool used to communicate with a Kubernetes cluster.
- **What is a Service?** A stable network endpoint for accessing a set of Pods.

Conclusion

A Kubernetes deployment and service were created and verified. Note: the local cluster requires a working container runtime.

Practical 6: Docker Containerization

Aim

Aim: To pull, run, inspect and stop Docker containers.

Software / Tools Required

Docker Desktop, Docker CLI, internet connection.

Procedure

1. Start Docker Desktop and wait until the Engine is running.

2. Verify Docker client and server using docker version or docker info.

3. Pull the hello-world image.

4. Run the image and verify the successful output.

5. List containers, including stopped containers.

6. Run an Nginx container in detached mode and map a host port.

7. Open the local URL in a browser, then stop and remove the container.

Commands / Configuration

```
docker --version
docker info
docker pull hello-world
docker run hello-world
docker ps
docker ps -a
docker run -d --name web-test -p 8080:80 nginx
docker ps
# Open http://localhost:8080
docker stop web-test
docker rm web-test
```

Expected Output / Result

hello-world prints a success message. docker ps -a lists containers. Nginx runs in the background and is available at http://localhost:8080 until stopped.

Troubleshooting: If the CLI says it cannot connect to the Docker API, open Docker Desktop and start the Engine. If Virtual Machine Platform was just enabled, restart Windows before retrying.

Important Viva Questions

- **What is Docker?** A platform for packaging and running applications in containers.
- **What is an image?** A read-only template used to create containers.
- **What is a container?** A runnable instance of an image.
- **What does docker pull do?** Downloads an image from a registry.
- **What is port mapping?** It connects a host port to a container port, for example 8080:80.

Conclusion

Docker images were downloaded and containers were run, inspected and stopped. The Docker Engine must be running for these commands to work.

Practical 7: Grafana Application Monitoring

Aim

Aim: To configure Grafana, connect a data source and create a monitoring dashboard.

Software / Tools Required

Grafana OSS, web browser; a data source such as Prometheus, or Grafana's built-in TestData source for a sample dashboard.

Procedure

1. Install Grafana OSS for Windows and start the Grafana service.

2. Open http://localhost:3000.

3. Sign in with the initial credentials shown by the installer/version (commonly admin/admin) and set a new password when prompted.

4. Open Connections/Data Sources and select Add data source.

5. Choose Prometheus and enter its server URL (commonly http://localhost:9090) if Prometheus is installed and running; select Save & Test.

6. Alternatively, use the built-in TestData data source to demonstrate panels without a Prometheus server.

7. Create a dashboard, add a visualization panel, select the data source, choose a sample query and save the dashboard.

Commands / Configuration

```
Grafana URL: http://localhost:3000
Prometheus URL (if locally installed): http://localhost:9090
Typical Prometheus data source URL: http://localhost:9090
For a simple sample: choose TestData data source and a random-walk/time-series scenario in a panel.
```

Expected Output / Result

Grafana opens on port 3000. The data source test succeeds when the source is reachable. The dashboard displays a graph or other selected visualization.

Note: Grafana does not generate metrics by itself. Prometheus must be installed and running for the Prometheus URL to work; use TestData for a quick demonstration.

Important Viva Questions

- **What is Grafana?** An open-source platform for querying, visualizing and monitoring data.
- **What is a data source?** A system from which Grafana retrieves data.
- **What is a dashboard?** A collection of panels used to display monitoring information.
- **What is a panel?** A single visualization, such as a graph or gauge.
- **What is Prometheus?** A monitoring and time-series database commonly used with Grafana.

Conclusion

A Grafana data source was configured and a dashboard with a visualization panel was created.

Practical 8: AWS Lambda Serverless Function

Aim

Aim: To create, deploy and test a serverless function using AWS Lambda.

Software / Tools Required

AWS account, browser, AWS Management Console, Python runtime.

Procedure

1. Sign in to AWS Management Console and open Lambda.
2. Select Create function, choose Author from scratch, enter a function name and select a supported Python runtime.
3. Create the function and open the Code source editor.
4. Replace the sample code with a Python Lambda handler.
5. Select Deploy to save the code.
6. Select Test, create a test event (an empty JSON object is sufficient for this example) and run it.
7. Inspect the execution result and logs.

Commands / Configuration

```
Example Python code:
def lambda_handler(event, context):
return {
'statusCode': 200,
'body': 'Hello World from AWS Lambda!'
}

Test event:
{}
```

Expected Output / Result

Execution succeeds and the response contains statusCode 200 and the Hello World message. The console also shows execution details and logs.

Important Viva Questions

- **What is AWS Lambda?** A serverless compute service that runs code in response to events.
- **What is serverless?** The cloud provider manages the underlying servers and scaling infrastructure.
- **What is a handler?** The function entry point invoked by the Lambda runtime.
- **What is an event?** The input data passed to a Lambda function.
- **What does Deploy do?** It saves the edited function code so it can be executed.

Conclusion

The Python Lambda function was deployed and tested successfully, returning an HTTP-style success response.

Last-minute Command Revision

Task	Command / Key
Git status	git status
Git stage + commit	git add . | git commit -m "message"
Git push	git push -u origin main
Jenkins local URL	http://localhost:8080
Docker image + run	docker pull hello-world | docker run hello-world
Docker containers	docker ps | docker ps -a
Kubernetes status	kubectl get deployments | kubectl get pods | kubectl get services
Grafana local URL	http://localhost:3000
AWS Lambda handler	def lambda_handler(event, context):

Exam tip: For every practical, remember the sequence: Aim → Tools → Procedure → Command/Configuration → Output → Conclusion. If an installation or service is unavailable during the exam, explain the required setup and show the commands/configuration clearly, subject to the lab rules.